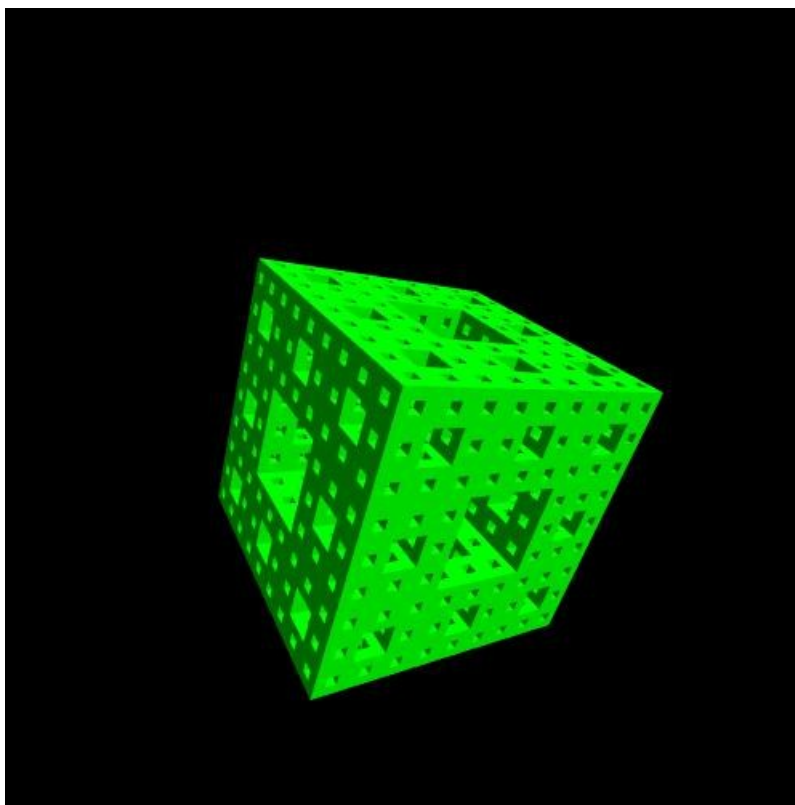




nom de votre projet :	The Graphics Engine
membres de l'équipe :	Arthur CALON
membres de l'équipe :	Corentin BUSSIÈRE
membres de l'équipe :	Robinson URSIN ALLIX
niveau d'étude :	terminale
établissement scolaire :	Lycée Gustave Eiffel (Dijon)
enseignant de NSI :	M. BELLIER

> PRÉSENTATION GÉNÉRALE :

Pouvez-vous présenter en quelques mots votre projet ?

Comment est né ce projet ? Quelle était la problématique de départ ?

Quels sont les objectifs ? À quels besoins répondez-vous ?

The Graphics Engine est un logiciel de dessin 2D et 3D original, disposant de fonctionnalités qui le distinguent des logiciels de dessin ordinaire. On peut s'en servir pour dessiner des fractales, créer des bandes dessinées, et même des animations de courte durée (au format GIF). Le logiciel est composé d'une partie pour la 2D, très légère, et d'une autre partie pour la 3D (permettant tout de même d'utiliser des fonctionnalités 2D), plus lourde, car elle dispose de plus de fonctionnalités.

Ce projet a été créé dans le but de concevoir un logiciel en rapport avec le thème « Arts et Informatique », tout en proposant quelque chose d'original. Après avoir cherché une idée, en cherchant à ne pas juste réinventer la roue, mais plutôt proposer quelque chose d'original, nous avons donc décidé de créer un logiciel de dessin disposant de fonctionnalités avancées, par exemple, un outil pour dessiner des fractales et un outil conçu pour les dessinateurs de bandes dessinées, on a appelé ce projet : « The Graphics Engine ».

Pour nous, la problématique était de créer un logiciel en rapport avec les thèmes « Arts et Informatique », qui soit original, et dont la création mette au défi nos capacités et nous permette de repartir avec beaucoup plus d'expérience qu'au départ, et en ayant développé de nouvelles compétences.

Les objectifs de ce projet sont : créer un logiciel de dessin disposant non seulement des fonctionnalités basiques d'un logiciel de dessin (crayons, formes) mais aussi des fonctionnalités avancées, que l'on ne retrouve pas dans d'autres logiciels de dessin ; écrire un code optimisé et portable, afin de minimiser la consommation de ressources du logiciel et maximiser le nombre de machines pouvant l'exécuter; écrire un code source lisible, convenablement commenté et documenté, afin que le code soit facile à maintenir et à améliorer ; et ainsi, proposer un logiciel de dessin open-source disposant des fonctionnalités avancées, disponible pour tous, sur plusieurs plateformes, et auquel tout le monde peut contribuer.

The Graphics Engine peut être utilisé pour créer des dessins numériques simples (avec des formes 2D ou 3D) ou avancés (contenant des fractales). Il peut également être utilisé comme un logiciel pour créer des bandes dessinées ou encore être utilisé pour faire de l'ASCII Art (art avec des caractères, ici, le logiciel transforme le dessin réalisé avec des formes en caractères). Il permet donc de faire de l'art, en utilisant un ordinateur, même si celui-ci ne dispose pas des composants de dernière génération.

> ORGANISATION DU TRAVAIL :

Pouvez-vous présenter chaque membre de l'équipe et préciser son rôle dans ce projet ?

Comment avez-vous réparti les tâches et pourquoi ?

Combien de temps avez-vous passé sur le projet ? Avez-vous travaillé en dehors de l'établissement scolaire ?

Quels sont les outils et/ou les logiciels utilisés pour la communication et le partage du code ?

Vous veillerez au bon équilibre des différentes tâches dans le groupe. Chaque membre de l'équipe doit impérativement réaliser un aspect technique du projet (hors design, gestion de projet).

Corentin BUSSIÈRE (17 ans), a découvert la programmation lors du dernier jour de son stage d'observation, en 3ème. Il a continué la programmation car il veut coder des jeux vidéos, et a donc appris, le Python, le Lua et le C#. Il travaille sur la partie 2D du projet.

Arthur CALON (17 ans), a commencé la programmation lorsqu'il était au collège. Après s'être rendu compte des limites des outils pour faire du noCode, il a décidé de se mettre à la programmation, alors qu'il ne faisait encore que des réparations sur les ordinateurs (et n'avait aucune expérience de la programmation). Arthur a appris les bases des langages HTML, JavaScript (pour le frontend, et le backend avec NodeJS), CSS, Solidity, Python, et plus tard Golang. Il développe surtout des applications web et utilise principalement HTML, JavaScript, CSS, NodeJS et Golang. Il travaille sur la partie 3D, la présentation et la documentation du projet.

Robinson URSIN ALLIX (16 ans) a découvert la programmation à l'âge de 8 ans avec scratch. Très vite, il s'est mis à faire de la programmation en utilisant les langages de programmation. Il code principalement en Python, mais maîtrise également l'HTML, le CSS, le Shell, le PHP, le C# et le Makefile. Il travaille sur la partie 3D du projet.

Après que Arthur et Robinson aient trouvé l'idée pour notre projet, nous avons décidé de le scinder en 2 grandes parties : une partie pour la 2D et une autre pour la 3D mais comprenant aussi des fonctionnalités de dessin 2D. Corentin a choisi de travailler sur la partie du projet gérant uniquement la 2D. Arthur et Robinson ont choisi de coder la partie qui gère la 3D, car c'est un défi de taille, et pour mieux relever ce défi, il vaut mieux s'y mettre à plusieurs, et bien s'organiser.

Nous avons passé plusieurs mois à développer ce projet, en codant parfois plus de 8 heures par semaine pour avancer sur celui-ci, notamment hors de l'établissement scolaire, parce que certaines fonctionnalités nécessitaient plusieurs heures pour être développées, testées et implémentées. De plus, le débogage d'un logiciel qui effectue du rendu 3D avec OpenGL et beaucoup plus long, plus dur, et consomme plus de ressources que celui d'un programme Python comme ceux que l'on code en NSI (liste chaînées en Python, algorithmes de tri, graphes, ...).

Pour partager les versions expérimentales du code, nous avons choisi d'utiliser un répertoire privé sur GitHub car nous avions déjà l'habitude d'utiliser cet outil, notamment pour publier nos projets personnels open-source. Nous avons choisi d'utiliser Discord pour communiquer car on peut y envoyer des images et vidéos légères facilement et cela permettait d'éviter d'avoir à payer un supplément pour s'envoyer des messages quand Arthur est chez son père, en Suisse. De plus Discord permet les appels et appels vidéos (avec partage d'écran), si on a en besoin. Plus précisément, on utilisait un salon dédié sur un des serveurs Discord d'Arthur, qui était dédié à la programmation, ce qui a plus tard permet la mise en place d'un 'early access' pour que des personnes ne faisant pas partie de l'équipe du projet puissent donner leur avis sur celui-ci, et aider à vérifier sa compatibilité avec différentes plateformes.

LES ÉTAPES DU PROJET :

Présenter les différentes étapes du projet (de l'idée jusqu'à la finalisation du projet)

Après avoir vu le thème « Arts et Informatique », nous avons cherché une idée en rapport avec ce thème. Nous avons donc eu l'idée d'un logiciel de dessin, puis on a cherché les fonctionnalités à ajouter que celui-ci se distingue des logiciels de dessin déjà existants.

Puis, on a commencé à coder de manière expérimentale pour sélectionner les modules à choisir. On a retenu les modules PyQt6 et Qtawesome pour la partie du logiciel qui fonctionne uniquement en 2D et glfw, PyOpenGL, trimesh, numpy pour la partie du logiciel qui fonctionne en 3D mais permet également de travailler en 2D.

Après cela, on a commencé à construire les bases du projet, les différents fichiers python se trouvaient tous dans le même répertoire. Ensuite, on a amélioré les différentes classes et fonctions, et ajouté des docstrings à celles-ci. Puis, on a modifié l'architecture du projet afin de ne plus avoir tous les fichiers python dans un seul répertoire. À ce moment, nous avons déjà implémenté les fonctionnalités de dessin de traits, sauvegarde et ouverture de projets pour la partie 2D ainsi que les fonctionnalités de dessin de formes, déplacement de la caméra, comptage d'images par secondes (fps) pour la partie 3D.

Après cela, nous avons implémenté la sauvegarde pour les images non-vectorielles, et les bases du chargement d'images non-vectorielles, en version expérimentale.

C'est à ce moment qu'on a commencé à rédiger la documentation, la présentation et le résumé du projet.

De plus, nous avons conçu la partie 3D de ce logiciel de façon à ce que l'image ne soit pas recalculée lorsque l'utilisateur n'effectue aucune action dessus, ce qui réduit fortement sa consommation de ressources (notamment de RAM, vRAM et cœurs du GPU dédiés à la 3D) en arrière-plan, cependant, en avant plan, le logiciel consomme peut consommer beaucoup des ressources disponibles, si l'utilisateur a choisi de ne pas activer vSync. Le taux d'utilisation CPU reste au dessous de 40 % et celui du GPU au dessous de 45 % sur la plupart des configurations testées en utilisant vSync. De plus, The Graphics Engine consomme généralement moins de 2 gigaoctets de RAM.

Ensuite, on est passé de pygame à glfw pour l'affichage du rendu 3D, à cause des problèmes de compatibilité causés par pygame, puis, on a ajouté la 1ère fractale 2D du logiciel : le flocon de Koch, cependant, le dessin d'une fractale précise peut ralentir le logiciel et augmenter drastiquement sa consommation de ressources (la précision de la fractale est décidée par l'utilisateur). À ce moment, on a mis en place un 'early access' pour que les membres du serveur Discord puissent utiliser The Graphics Engine, pas encore rendu public, car le code nécessitait encore beaucoup d'améliorations.

Après cela, on a ajouté la possibilité de dessiner les cercles et corrigé le calcul des vertices des étoiles.

Puis, on a refait l'interface utilisateur de la partie 3D, afin de la rendre plus intuitive, et d'améliorer son design.

Ensuite, on a implémenté de nombreuses fonctionnalités pour la sauvegarde du dessin dans la partie 3D.

Le vendredi 7 mars, on a uploadé la première version (publique) de The Graphics Engine sur GitLab. Dans la soirée, et pendant la journée du 8 mars, on a corrigé les bugs que nous n'avions pas encore repérés avant et légèrement amélioré certains menus de l'interface utilisateur de la partie 3D.

Le mardi 25 mars, nous avons terminé les améliorations du projet, et terminé le dossier technique.

> FONCTIONNEMENT ET OPÉRATIONNALITÉ :

Pouvez-vous présenter l'état d'avancement du projet au moment du dépôt ? (ce qui est terminé, en cours de réalisation, reste à faire)

Quelles approches avez-vous mis en œuvre pour vérifier l'absence de bugs et garantir une facilité d'utilisation de votre projet ?

Quelles sont les difficultés rencontrées et les solutions apportées ?

Le projet dispose de nombreuses fonctionnalités, cependant, le mode bande dessinée, le menu d'édition de formes, et le multi caméra pourraient être améliorés.

Afin de vérifier l'absence de bugs, on a testé chacune des fonctions après leur création et après chaque modification apportée, et dans plusieurs cas, puis amélioré le code de façon à ne plus rencontrer d'erreurs lors de l'exécution du code. Lorsqu'un bug persistait, nous discutons sur Discord et cherchions l'origine de celui-ci, avant de modifier le code, et ainsi le corriger.

La quantité de code dans les fichiers fait partie des difficultés rencontrées, nous avons donc divisé le code en plusieurs classes, réparties dans plusieurs fichiers et nous avons utilisé le système de Directory as module de Python 3 afin d'éviter d'avoir trop de fichiers dans un seul dossier.

Nous avons ajouté des docstrings et des commentaires, que l'on a progressivement amélioré afin de rendre le code plus facile à maintenir. Une autre des difficultés était la consommation de ressources du logiciel, notamment lors du rendu 3D, pour remédier à cela, nous avons optimisé le code du projet, chaque fois où nous le pouvions. Le fonctionnement de PyOpenGL et d'OpenGL comportant certaines particularités, notamment avec l'utilisation des buffers, nous avons dû, à plusieurs reprises modifier du code de notre logiciel pour éviter de réallouer inutilement la RAM et de trop en consommer. Au début du projet, on utilisait pygame avec OpenGL, ceci s'est avéré posé des problèmes de compatibilité sous Linux, on a donc choisi d'utiliser glfw à la place, car ce module est plus adapté et légèrement plus performant dans le contexte de l'utilisation d'OpenGL.

> OUVERTURE :

Quelles sont les nouvelles fonctionnalités à moyen terme ? Avez-vous des idées d'amélioration de votre projet ? Pourriez-vous apporter une analyse critique de votre projet ? Si c'était à refaire, que changeriez-vous dans votre organisation, les fonctionnalités du projet et les choix techniques ?

Quelles compétences/appétences/connaissances avez-vous développé grâce à ce concours ?

En quoi votre projet favorise-t-il l'inclusion ?

À moyen terme, la possibilité de dessiner plus de formes, de fractales et une amélioration du chargeur de mods intégré comme la possibilité de charger des mods pendant que le logiciel est en cours de fonctionnement (The Graphics Forge) peut être envisagé. Une idée d'amélioration du projet serait d'utiliser customtkinter pour un meilleur design de l'interface utilisateur de la partie 3D.

Bien que The Graphics Engine soit très optimisé, l'interface utilisateur de la partie 3D, et, l'interopérabilité entre la partie 2D et la partie 3D peuvent être améliorées, de plus le dessin de fractales peut s'avérer causer des ralentissements (notamment lorsque vSync est désactivé) sur certaines configurations.

Si on devait refaire ce projet, on utiliserait, dès le départ, ttk (de tkinter) pour l'interface utilisateur de la partie 3D, travaillerait plus sur l'interopérabilité entre la partie 2D et la partie 3D (ou abandonnerait complètement l'idée d'une partie 2D indépendante de la partie 3D), et ajouterai des outils de débogage avancé pour simplifier cette tâche, parfois lourde, surtout en travaillant avec des modules chargés d'afficher des éléments à l'écran (comme PyOpenGL).

Compte tenu de la difficulté du projet, et du temps requis pour résoudre certaines bugs, tous les membres devaient collaborer et s'entraider sur le développement de nouvelles fonctionnalités et l'amélioration de celles déjà existantes, ce qui fait que personne ne peut être écarté du projet, bien que les membres de l'équipe aient des centres d'intérêts parfois très différents. En plus de cela, on ouvrira la possibilité à toute personne intéressée (sans prérequis) de contribuer à The Graphics Engine après le concours (dès le 25 juin 2025, afin d'avoir le temps de mettre en place une politique de contribution) et on fournira des astuces liées aux difficultés que l'on a rencontrées lors du développement du logiciel, afin de favoriser la portée de la possibilité de contribuer à la

maintenance et l'amélioration de The Graphics Engine.

Ce document est l'un des livrables à fournir obligatoirement lors du dépôt de votre projet : 4 pages maximum. Le non-respect du modèle fourni peut impacter la notation.

La documentation technique complète est à intégrer dans le dossier technique, dans un répertoire nommé doc.

Pour accéder à la liste complète des éléments à fournir, consultez la page **[Comment participer ?](#)**.

Vous avez des questions sur le concours ? Vous souhaitez des informations complémentaires pour déposer un projet ?

Contactez-nous à **info@trophees-nsi.fr** ou consultez la page **[Foire aux questions](#)**.